University of Petra

307307
Generative AI

Introduction to Agentic AI

Introduction to Agentic AI

- Large Language Models (LLMs) are very good at producing text: answering questions, summarizing, drafting, and explaining. But on their own, they behave like a “single interaction” system: you ask once, they answer once.
- Many real tasks do not fit that shape. For example:
  - **Up-to-date facts** (the model may not know what happened yesterday).
  - **Precise computation** (the model can make arithmetic or logic mistakes).
  - **Verification** (the model cannot reliably prove it is correct).
  - **Multi-step coordination** (the model may skip steps or lose track).
  - **Interaction with systems** (files, APIs, databases, code, tickets, emails).
- These tasks require more than writing text. They require **doing steps**, **checking results**, and sometimes **using external systems** (like search engines, files, APIs, or code execution).
- **Agentic AI** is the approach of building AI that can handle this kind of work reliably: not only generating language, but also managing a multi-step process toward a goal.

What We Mean by “An Agent” (Plain Definition)

- An **agent** is an AI system that is **given a goal** and can make progress toward that goal by deciding what to do next (Planning).
- In simple terms, an agent is like an assistant that can:
  - read your instruction,
  - decide the next step,
  - and continue step-by-step until it reaches a usable result.
- The difference from a normal chatbot is not “intelligence.” The difference is **behavior over time**:
  - A chatbot gives an answer and stops.
  - An agent keeps working until the goal is achieved, or it decides it cannot proceed.
- This does not mean the agent is “independent” or “uncontrolled.” In well-designed systems, the agent operates inside boundaries: limited permissions, limited tools, and clear stopping conditions.

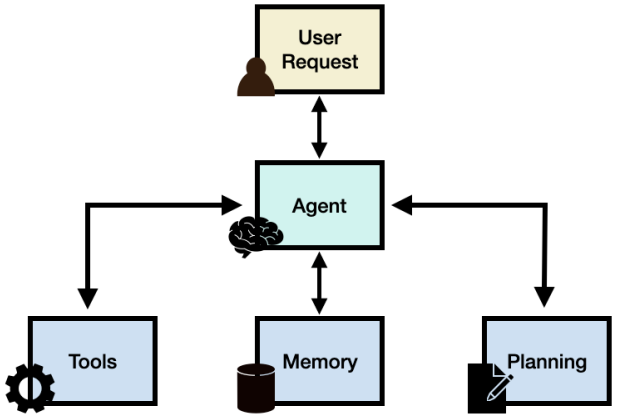
The Building Blocks of Agentic AI

Agentic AI is not a single model feature. It is a system built from a few common building blocks:

1) The Language Model (the brain)
This is the component that understands instructions, generates plans, and makes decisions in natural language.

2) Tools (the hands)
Tools are external capabilities the agent can use to do real work, such as:

- searching the web,
- calling an API,
- reading a file,
- running Python code,
- querying a database.



The Building Blocks of Agentic AI

3) State / Working Context (the scratchpad)
To do multi-step work, the agent needs a place to keep track of:

- what it already tried,
- what it learned,
- intermediate results,
- what still needs to be done.

4) Control Rules (the boundaries)
Real agents must follow rules such as:

- what tools are allowed,
- when to stop,
- when to ask for clarification,
- what counts as “done.”

Looking Inside an Agentic System

- So far, we talked about *what agents do*.
  Now we briefly look at **what they are made of**.
- An agentic AI system is not a single block. It is a **small system of components**, each with a clear responsibility. Thinking in components helps us understand:
  - why agents behave the way they do,
  - where errors come from,
  - and how to improve reliability.
- At a high level, most agentic systems are built from four parts:
  - a language model,
  - tools,
  - memory or state,
  - and a controller.
- Each part solves a different problem.

The Language Model: Reasoning and Decision Core

- The language model is the **reasoning engine** of the agent.
- Its main responsibilities are:
  - understanding the goal or instruction,
  - interpreting the current situation,
  - deciding what to do next,
  - and explaining results in natural language.
- Importantly, the model is **not responsible for executing actions**.
  It does not search the internet, run code, or access files by itself.
- Instead, the model *decides*:
  - whether an action is needed,
  - which action to take,
  - and how to use the result once it comes back.
- You can think of the language model as the *brain*, not the hands.

Why Tools Matter: Thinking vs Doing

- Language models are good at **thinking and explaining**, but they are weak at **doing real-world actions**.
- For example:
  - They can describe how to calculate statistics, but may make mistakes doing it.
  - They can talk about search results, but cannot actually browse the internet on their own.
- Agentic systems solve this by separating roles:
  - The language model **decides what should be done**.
  - A tool **does the actual work**.
- Examples of tools:
  - Web search tools for up-to-date information.
  - Python or calculator tools for exact computation.
  - File readers for documents and datasets.
  - APIs for external systems.
- This separation dramatically improves reliability, because the agent no longer needs to “pretend” it did the work.

Tools: How Agents Do Real Work

- Tools are how agents interact with the real world.
- A tool is any external capability that performs an action the model cannot reliably do on its own, such as:
  - searching the web,
  - querying a database,
  - reading files,
  - executing code,
  - calling an API.
- The key idea is separation:
  - the model **chooses** the tool and provides inputs,
  - the tool **executes** the action and returns results,
  - the model then **reasons over the result**.
- This separation is critical.
  It prevents the model from guessing and allows the system to rely on deterministic, verifiable operations.

Memory and State: Keeping Track of Progress

- For an agent to work across multiple steps, it must remember what has already happened.
- This memory does not have to be complicated. At a minimum, the agent needs:
  - the original goal,
  - the results of previous steps,
  - and any intermediate findings.
- This information is often stored as **state**:
  a structured record of what the agent knows so far.
- Without state, every step would feel like starting over.
  With state, the agent can build on previous work, avoid repetition, and make better decisions.

The Controller: Rules and Boundaries

- The controller is the part of the system that **manages behavior**.
- It decides things like:
  - when the agent should stop,
  - how many steps are allowed,
  - which tools are permitted,
  - and what happens if something fails.
- This is where safety and reliability live.
- The controller ensures that the agent:
  - does not run forever,
  - does not exceed permissions,
  - and operates within defined limits.
- In practice, this is what turns an agent from a risky experiment into a usable system.

A Simple Example: “Find, Compute, Explain”

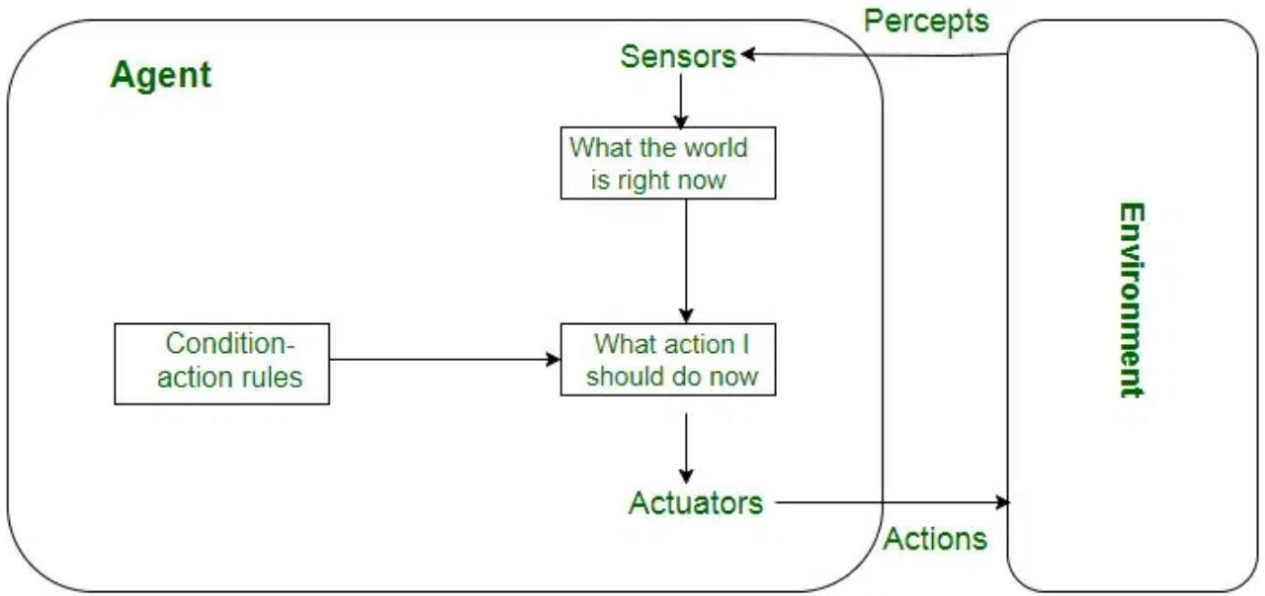
To make the idea concrete, consider a task that looks easy but actually has multiple parts:

- **Task:** “Find the population of Amman, compute what 3% of it is, and explain what that number means.”
- A normal LLM might:
  - guess a population number,
  - do a rough calculation,
  - produce a confident explanation.
- An agentic system would treat this as a process:
  - **Find the population from a reliable source** (search tool).
  - **Compute 3% precisely** (calculator or Python tool).
  - **Write the explanation** (LLM).
  - **Include the source and the computed result**.
- The key idea: the writing part is only the final step. The correctness comes from using the right tool at the right time, and keeping track of results.

The “Agent Workflow”

When an agent solves a task, it usually follows a repeated pattern:

- It reads the goal.
- It decides (PLANS): “What is the next step that moves me forward?”
- If needed, it uses a tool to get information or perform an action.
- It looks at the result and decides what to do next.



This repeats until one of these outcomes happens:

- The goal is achieved (a usable final result).
- The agent needs clarification (missing information).
- The agent is blocked (no permission, no tool, or unreliable data).

This is the practical meaning of “agents work step-by-step.” It is not magic. It is structured progress.

How an Agent Decides “What to Do Next”

- When an agent is working on a task, it constantly faces a simple question:
- **“What is the best next step right now?”**
- This decision is not random. The agent looks at:
  - the original goal,
  - what has already been done,
  - and what information or results are missing.
- Instead of jumping directly to a final answer, the agent breaks the task into **smaller, manageable steps**. Each step is chosen because it moves the agent closer to completing the goal.
- This ability to reason about intermediate steps is what allows agents to handle complex tasks that would confuse a single prompt.

Step-by-Step Reasoning (Chain of Thought)

- One of the most important ideas in agentic AI is **making reasoning explicit**.
- Rather than silently guessing an answer, the agent:
  - writes down intermediate thoughts,
  - explains why a certain step is needed,
  - and uses that reasoning to choose actions.
- This is commonly called **Chain-of-Thought reasoning**.
- In practice, this means:
  - “First, I need to understand the problem.”
  - “Next, I need to gather missing information.”
  - “Then, I need to compute or verify something.”
  - “Finally, I can produce a clear explanation.”
- By reasoning step-by-step, agents are:
  - more accurate,
  - easier to debug,
  - and less likely to skip important steps.

Checking and Correcting: Reflective Agents

- Even with good reasoning and tools, mistakes can happen.
- A key improvement in agentic systems is **reflection**:
  the ability of the agent to look at its own output and ask:
  - “Does this make sense?”
  - “Is this result consistent with the goal?”
  - “Did I miss something important?”
- A reflective agent may:
  - revise its plan,
  - repeat a step using a different approach,
  - or improve the final explanation.
- This is how agents move from “first attempt” answers to **usable, higher-quality results**.
- Reflection turns agents from one-shot responders into systems that can **self-correct**.

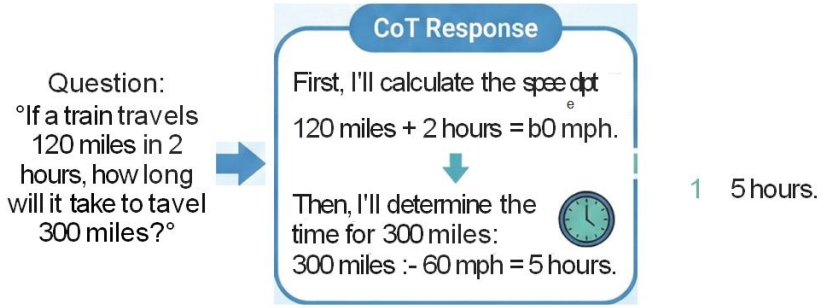
Chain-of-Thought (CoT): The Foundation

The simplest approach where the AI breaks down complex problems into a sequential series of reasoning steps, much like showing your work in math class.

The AI explicitly articulates 1s reasoning process xep-by-step before arriving at a a final answer. Each step logically follows from the previous one in a linear chain.

HOW IT WORKS: STEP-BY-STEP REASONING

HOW IT WORKS: TRAIN SPEED **CALCULATION** EXAMPLE



BEST FOR

- Mathematical problems.
- Logical reasoning.
- Straightforward analysis where one step leads clearly to the next.

Putting It Together: A Simple Agent Pattern

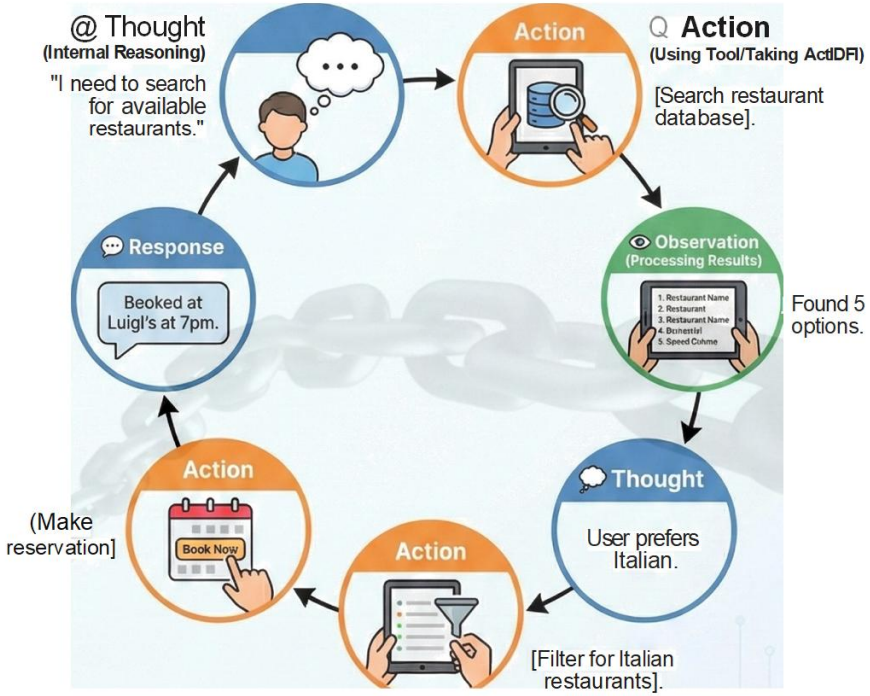
- At this point, we can summarize a very common agent pattern:
  1. Read the goal.
  2. Reason about the next step.
  3. Use a tool if needed.
  4. Observe the result.
  5. Reflect and decide what to do next.
  6. Stop when the goal is satisfied.
- This pattern appears again and again in real systems:
  - data analysis agents,
  - research assistants,
  - automation workflows,
  - code analysis tools.
- The important takeaway is not the terminology, but the mindset:
  agentic AI is about structured progress toward a goal, not clever text generation.

ReAct (Reasoning + Acting): The Interactive Approach

Combines reasoning with action-taking, interacting with external tools to execute tasks.

The AI alternates between three modes: Thought (internal reasoning), Action (using a tool or taking an action), and **Observation** (processing the results). This cycle repeats until the task is complete.

HOW IT WORKS: RESTAURANT BOOKING EXAMPLE



BEST FOR

- Tasks requiring externat data.
- Multi-step processes involving tool use.
- Real-world problem-solving where actions aPect outcomes.

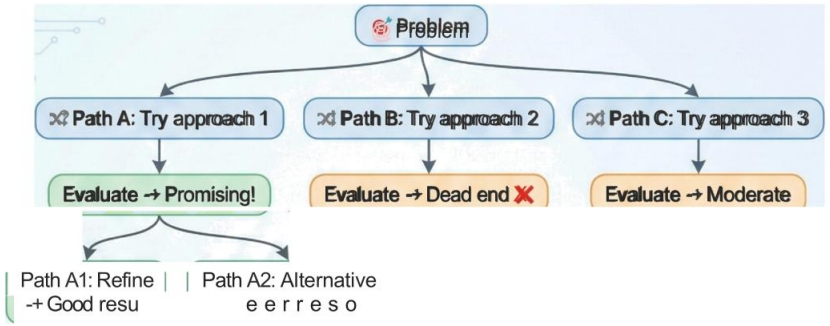
BUSINESS INTELLIGENCE AND
DATA ANALYTICS

Tree-of-Thoughts (ToT): The Exploratory Approach

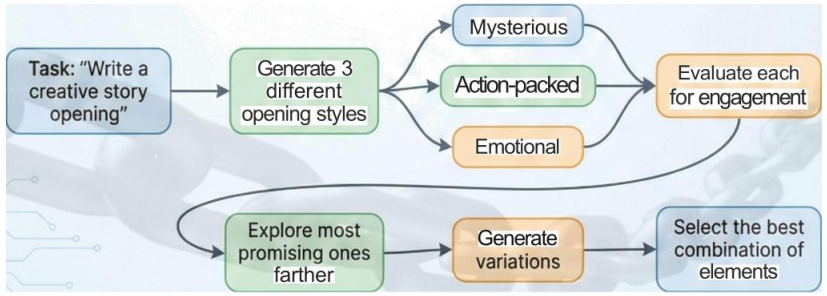
Allows the AI to explore multiple reasoning paths simultaneously, like a chess player considering several possible moves before choosing the best one.

Instead of following a single chain, the AI generates multiple potential reasoning paths (branches), evaluates them, and can backtrack if a path seems unproductive. It explores the "tree" of possibilities to find the optimal solution.

HOW IT WORXS: EXPLORINC MULTIPLE PATHS



HOW IT WORKS: **CREATIVE STORY OPENING EXAMPLE**

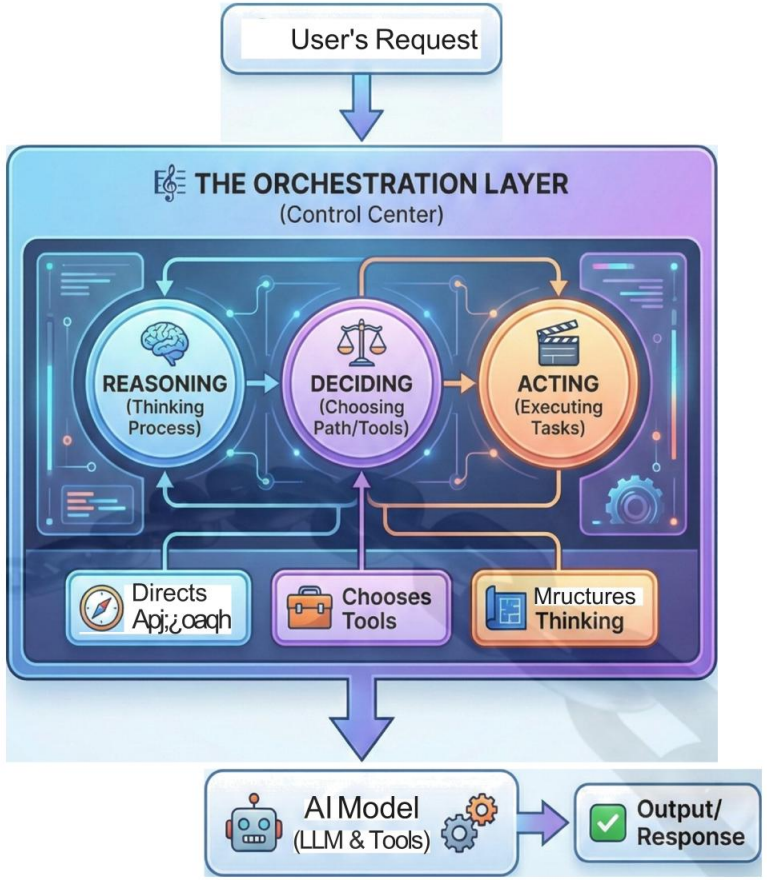


BEST FOR

- Creative tasks.
- Complex problem-solving where multiple approaches should be considered.
- Strategic planning.
- Optimization problems.

The Orchestration Layer: AI Control Center

The central hub that manages how an AI agent reasons, decides, and acts, sitting between the user and the model.

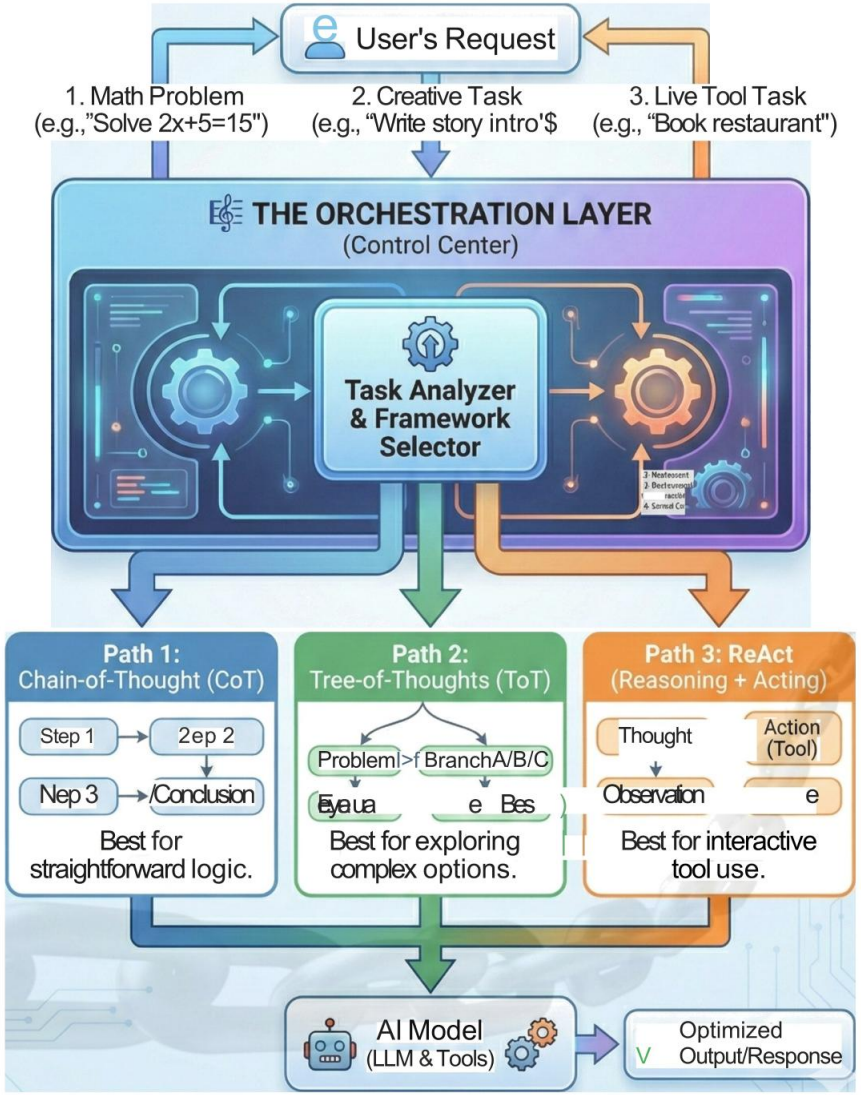


BENEFITS OF CHOOSING THE RIGHT FRAMEWORK

- More Capable AI Agents.
- More Efficient Operations.
- More Reliable Performance.

Orchestration Layer in Action: Framework Selection Example

How the control center dynamically chooses the best reasoning path (CoT, ToT, ReAct) based on the specific user task.



Key DiXerences at a Glance: CoT, ReAct, ToT

A comparison of AI reasoning methods based on structure, complexity, speed, and use.

	Chain-of-Thought (CoT)	ReAct (Reasoning + Acting)	Tree-of-Thoughts (ToT)
Structure	Linear, sequential	Iterative, action-based	Branching, exploratory
Complexity	Simple to moderate	Moderate to high	High
Speed	Fast	Variable (depends on actions)	Slower (explores multiple paths)
Use	Straightforward reasoning	Interactive tasks with tools	Complex, creative problems

BEST FOR

- CoT: Mathematical & logical problems.
- ReAct: Tasks needing external data & tools.
- ToT: Strategic planning & creative solutions.

Simple Example

Code: Setup + Agent Decision

```
import pandas as pd
import google.generativeai as genai
import json

# Load dataset
df = pd.read_csv("sales.csv")   # Example dataset

# Configure Gemini
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-flash")

# Tools the agent can use
def summary_tool(_):
    return df.describe().to_string()

def correlation_tool(_):
    return df.corr(numeric_only=True).to_string()

tools = {
    "summary": summary_tool,
    "correlation": correlation_tool
}
```

Code: Setup + Agent Decision

```
# Agent decides next action
def agent_decide(state):
    prompt = f"""
    You are a data analyst agent. Based on the task and dataset preview below,
    choose the best next analytical action.

    Return JSON only:
    {{"action": "..."}}

    State:
    {state} """
    return model.generate_content(prompt).text
```

Code: Execute Tool + Final Interpretation

```
def execute(action_json):
    data = json.loads(action_json)
    action = data["action"]


    if action in tools:
        return tools[action](None)
    else:
        return "Unknown action"


# Initial state includes task + preview of data
state = f"""
Goal: Provide insights about the sales dataset.
Data preview:
{df.head().to_string()}
"""
```

Code: Execute Tool + Final Interpretation

```
# Step 1: Agent chooses action
decision = agent_decide(state)
print("Model decision:", decision)

# Step 2: Python executes tool
observation = execute(decision)

# Update state
state += f"\nExecuted: {decision}\nObservation:\n{observation}"

# Step 3: Agent provides final insights
final_prompt = f"""
You are a senior data analyst.
Based on the updated state, produce a short, clear explanation of the insights.
"""

final_answer = model.generate_content(final_prompt).text

print("\nFinal Insight Report:\n", final_answer)
```